📘 Assignment: Student Course Management System

🎯 Objective

Build a **Student Course Management System** using **ASP.NET Core MVC + Entity Framework Core** implementing:

- Code First & DB First approaches
- Entity relationships
- CRUD operations
- Dependency Injection

🧩 Module 1: Project Setup

Tasks:

1. Create a new **ASP.NET Core MVC Application**
2. Add required NuGet packages:
    - EF Core
    - SQL Server Provider
3. Configure **connection string** in appsettings.json
4. Setup **DbContext class**

🧱 Module 2: EF Core Code First Approach

Scenario:

Design a system with:

- Student
- Course
- Enrollment

Tasks:

1. Create Models:
    - Student (Id, Name, Email)
    - Course (Id, Title, Credits)
    - Enrollment (Id, StudentId, CourseId, Grade)
2. Define relationships:
    - One Student → Many Enrollments
    - One Course → Many Enrollments
3. Configure relationships using:
    - Data Annotations OR Fluent API
4. Run migrations:
    - Add Migration
    - Update Database

🔗 Module 3: Entity Relationships

Tasks:

1. Implement:
    - One-to-Many (Student → Enrollments)
    - Many-to-Many (Student ↔ Course via Enrollment)
2. Display relational data:
    - Show Student with enrolled courses
    - Show Course with student list
3. Use:
    - Include()
    - ThenInclude()

🔄 Module 4: CRUD Operations

Tasks:

Implement full CRUD for:

1. Student
2. Course
3. Enrollment

Requirements:

- Create forms using **Tag Helpers**
- Validate inputs using **Model Validation**
- Display data using **Views**
- Use **Scaffold OR manual implementation**

🗂️ Module 5: EF Core Database First Approach

Scenario:

Use an existing database.

Tasks:

1. Create a database manually in SQL Server:
    - Tables: Department, Teacher
2. Run Scaffold command to generate:
    - Models
    - DbContext
3. Perform CRUD operations using generated models
4. Compare:
    - Code First vs DB First (write 5 differences)

💉 Module 6: Dependency Injection

Tasks:

1. Create Repository Interfaces:
    - IStudentRepository
    - ICourseRepository
2. Implement repositories:
    - StudentRepository
    - CourseRepository
3. Register services in Program.cs:
    - Scoped lifetime
4. Inject repositories into Controllers

📊 **Expected Output**

- Functional MVC application
- Database with proper relationships
- CRUD operations working for all entities
- Clean architecture using DI